

Interpolación Geométrica de Contornos de Gliomas entre Slices de Resonancia Magnética

Abel Albuez, Santiago Gil Gallego y Jesús Puenayan
Maestría en Ingeniería de Sistemas y Computación
Pontificia Universidad Javeriana, Bogotá D.C., Colombia
Profesor: Leonardo Flórez-Valencia Geometría Computacional 2026
Repositorio: <https://github.com/AbelAlbuez/computational-geometry-cgal>

Resumen—Este documento presenta el desarrollo e integración de un pipeline híbrido para la interpolación de contornos 2D de tumor activo (*Enhancing Tumor*, ET) extraídos de cortes axiales de resonancia magnética cerebral (dataset público *BraTS 2024 GLI*). La arquitectura se compone de dos enfoques analizados de forma comparativa. En primer lugar, un pipeline paramétrico en C++17/20 sobre CGAL que encadena un remuestreo por longitud de arco, una alineación topológica por ángulo polar respecto al centroide, una interpolación lineal directa y un módulo de resolución de auto-intersecciones mediante el algoritmo de Bentley-Ottmann, culminando en la reconstrucción volumétrica multicapa. En segundo lugar, se introduce la integración de un modelo neuronal de línea base (*baseline*) fundamentado en estimación de flujo óptico bidireccional (*Video Frame Interpolation*, VFI). La evaluación cuantitativa y visual del sistema demuestra que, mientras el enfoque puramente geométrico garantiza precisión sub-milimétrica estricta en las fronteras anatómicas, la inferencia neuronal resulta cualitativamente superior y más robusta ante alteraciones topológicas masivas entre cortes, evidenciando la complementariedad de ambas metodologías.

Index Terms—interpolación de contornos, geometría computacional, CGAL, Bentley-Ottmann, BraTS, resonancia magnética, gliomas

I. INTRODUCCIÓN

La reconstrucción tridimensional de tumores cerebrales a partir de secuencias de resonancia magnética (MRI) es una tarea relevante para el diagnóstico, la planeación quirúrgica y el seguimiento clínico de pacientes con gliomas. Una estrategia extendida consiste en segmentar slice por slice la región de interés y luego *interpol* los contornos 2D obtenidos para reconstruir una superficie cerrada en 3D.

Este trabajo utiliza el dataset *BraTS 2024 GLI* [4] (Brain Tumor Segmentation Challenge — Adult Glioma), que provee volúmenes de MRI con segmentaciones expertas. De cada volumen se extrae el contorno del *Enhancing Tumor* (label ET = 3) por slice axial mediante *scikit-image*, obteniendo polígonos cerrados almacenados en formato *.obj*.

La motivación última proviene de la línea de investigación del autor sobre segmentación automática de gliomas: una vez segmentado el tumor en slices 2D, se requiere un módulo geométrico que produzca slices intermedios consistentes — sin auto-intersecciones — para alimentar etapas posteriores de reconstrucción volumétrica.

II. MARCO TEÓRICO

II-A. Interpolación entre slices paralelos

Barequet y Sharir [1] formalizan el problema de interpolación lineal por tramos entre cortes poligonales paralelos como un problema de *correspondencia* entre vértices: dadas dos poligonales A y B en planos paralelos, construir una superficie triangulada que las una. Su trabajo establece el marco bajo el cual opera el módulo aquí descrito, restringido al caso $|A| = |B|$ (igualando los vértices mediante *resampling*) y a la interpolación lineal directa de vértices correspondientes.

II-B. Interpolación en imágenes médicas

Meijster y Roerdink [2] comparan métodos de interpolación (vecino más cercano, lineal, cúbica, basada en forma) en el contexto de procesamiento de imágenes médicas. Concluyen que la interpolación lineal por vértices ofrece una relación calidad/complejidad adecuada para contornos densamente muestreados, justificando su uso como *baseline* en este módulo antes de considerar variantes basadas en forma o en transformadas de distancia.

II-C. Polígonos auto-intersecantes

Souviron [3] presenta un esquema de detección y corrección de auto-intersecciones en polígonos cerrados con memoria mínima. En este trabajo se adopta una estrategia equivalente en el espíritu pero apoyada en CGAL: se detectan los cruces con el barrido de Bentley-Ottmann y se descompone el polígono en sub-lazos, reteniendo el de mayor área. El algoritmo de Bentley-Ottmann en sí está implementado dentro de la librería *pujCGAL* provista por el profesor del curso.

III. METODOLOGÍA — MÓDULO DE ABEL

El módulo está estructurado como una cadena de clases independientes sobre el kernel `CGAL::Exact_predicates_inexact_constructions_kernel` y el tipo `TContour = std::vector<Point_2>`. Las responsabilidades se descomponen en cuatro etapas, cada una encapsulada en una clase con un único método estático.

III-A. Resampling por longitud de arco (ContourResampler)

Dado un contorno cerrado $C = \{p_0, p_1, \dots, p_{m-1}\}$, se calcula la longitud de arco acumulada

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{(i+1) \bmod m} - p_i\|, \quad i = 0, \dots, m-1, \quad (1)$$

con $L = s_m$ la longitud total del contorno cerrado.

Para producir un contorno con exactamente n vértices uniformemente distribuidos por longitud de arco se evalúa, para cada $k \in \{0, \dots, n-1\}$, el parámetro $s_k^* = kL/n$, se localiza el segmento $[p_i, p_{i+1}]$ tal que $s_i \leq s_k^* \leq s_{i+1}$, y se interpola

$$u = \frac{s_k^* - s_i}{s_{i+1} - s_i}, \quad q_k = (1 - u)p_i + up_{(i+1) \bmod m}. \quad (2)$$

Esta etapa garantiza la precondition $|A| = |B|$ requerida por la interpolación lineal.

III-B. Interpolación lineal (LinearInterpolator)

Sean $A = \{a_0, \dots, a_{n-1}\}$ y $B = \{b_0, \dots, b_{n-1}\}$ dos contornos con el mismo número de vértices (tras el *resampling*). El contorno interpolado $P(t)$ para $t \in [0, 1]$ se define componente a componente:

$$P_i(t) = (1 - t)a_i + tb_i, \quad i = 0, 1, \dots, n-1. \quad (3)$$

En particular, $t = 0,5$ produce el slice intermedio entre A y B .

```

1 for( std::size_t i = 0; i < n; ++i )
2 {
3     const double ax = CGAL::to_double( A[i].x() );
4     const double ay = CGAL::to_double( A[i].y() );
5     const double bx = CGAL::to_double( B[i].x() );
6     const double by = CGAL::to_double( B[i].y() );
7     out.emplace_back( (1.0 - t)*ax + t*bx,
8                       (1.0 - t)*ay + t*by );
9 }

```

Listing 1. Núcleo de la interpolación lineal.

III-C. Detección de auto-intersecciones (SelfIntersectionResolver)

El contorno interpolado se convierte primero en un vector de segmentos CGAL conectando vértices consecutivos en orden circular:

$$S_i = \overline{P_i P_{(i+1) \bmod n}}, \quad i = 0, \dots, n-1. \quad (4)$$

A continuación se invoca el algoritmo de barrido Bentley–Ottmann [3] provisto por `pujCGAL`:

```

1 std::vector< TPoint > inters;
2 pujCGAL::SegmentsIntersection::BentleyOttmann(
3     segs.begin(), segs.end(),
4     std::back_inserter( inters )
5 );

```

Listing 2. Invocación del barrido Bentley–Ottmann de `pujCGAL`.

Se filtran las intersecciones que coinciden con un vértice original del contorno (extremos compartidos entre aristas consecutivas, que no representan cruces reales). Si el conjunto resultante es vacío, el contorno se retorna inalterado.

III-D. Resolución de cruces

Cuando existen cruces, cada punto de intersección p_j se asocia a las dos aristas que lo contienen y se calcula el parámetro $t \in [0, 1]$ de su posición a lo largo de cada arista. Esto permite construir una *traversal subdividida*: una lista ordenada que recorre el contorno insertando los puntos de cruce en su posición correspondiente.

Sobre esta traversal se aplica un esquema clásico de extracción de lazos mediante pila: cuando un punto de intersección aparece por segunda vez se cierra un sub-lazo con todos los nodos apilados desde su primera aparición. Cada sub-lazo L_k tiene un área absoluta dada por la fórmula del polígono (*shoelace*):

$$\text{Area}(L_k) = \frac{1}{2} \left| \sum_{i=0}^{|L_k|-1} (x_i y_{i+1} - x_{i+1} y_i) \right|. \quad (5)$$

El módulo retiene el sub-lazo L_{k^*} con $k^* = \arg \max_k \text{Area}(L_k)$, es decir, el lazo exterior de mayor extensión, descartando lazos parásitos típicos cuando los contornos fuente difieren mucho en forma.

III-E. Alineación Topológica y Línea Base Neuronal (Aportes de Jesús Alberto)

Para complementar el enfoque geométrico y establecer un punto de comparación, se desarrolló una integración con un modelo de aprendizaje profundo de línea base (*baseline*) basado en *Video Frame Interpolation* (VFI).

Previo a cualquier interpolación, se implementó un algoritmo de alineación por ángulo polar respecto al centroide. Esto garantiza una correspondencia topológica uno a uno entre los vértices de los contornos A y B , evitando el retorcimiento (*twisting*) de la malla antes de llegar al resolver de Bentley–Ottmann.

El flujo neuronal consiste en rasterizar los contornos `.obj` a máscaras 2D con *padding* dinámico, inferir el flujo óptico bidireccional mediante el modelo VFI para predecir el estado intermedio ($t = 0,5$), y reconstruir la geometría continua mediante *Marching Squares*.

IV. RESULTADOS

IV-A. Configuración experimental

La evaluación se realizó sobre el dataset *BraTS 2024 GLI* restringido al label *Enhancing Tumor* (ET, valor 3). Los contornos se extrajeron por slice axial mediante `scikit-image` y se serializaron en formato `.obj` 2D. Sobre este corpus se ejecutaron **4 pares representativos** de slices con parámetro de interpolación fijo $t = 0,5$, cada uno con una intención distinta:

1. slices consecutivos del mismo caso (variación morfológica mínima);
2. slices separados por 6 cortes del mismo caso (variación moderada);
3. par extremo (primer y último slice con ET) dentro de un mismo caso (variación máxima intra-caso);
4. par cruzado entre dos casos distintos (primer slice del caso X contra primer slice del caso Y).

Sobre la versión inicial del interpolador se incorporaron **tres mejoras geométricas** de pre-alineación (orientación, centroide y rotación cíclica óptima del punto de inicio), descritas en la Sección IV-B. Cada ejecución produce un `.obj` 3D y una visualización PNG con los dos contornos fuente y el contorno interpolado superpuestos.

IV-B. Mejoras geométricas implementadas

IV-B1. Normalización de orientación (CCW): La interpolación lineal vértice a vértice es *sensible al sentido de recorrido*: si A está en sentido CCW y B en CW, la correspondencia $A_i \leftrightarrow B_i$ recorre los contornos en direcciones opuestas y produce un polígono retorcido. Para evitarlo se calcula el área con signo de cada contorno mediante la fórmula *shoelace*:

$$\text{area}(C) = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i), \quad (6)$$

y si $\text{area}(C) < 0$ se invierte el orden de los vértices, garantizando que ambos contornos sean CCW antes de interpolar.

IV-B2. Alineación por centroide: La interpolación lineal implícitamente promedia las posiciones absolutas. Para que la geometría intermedia no quede distorsionada por una traslación entre A y B , se calculan los centroides

$$\bar{c}_A = \frac{1}{|A|} \sum_i A_i, \quad \bar{c}_B = \frac{1}{|B|} \sum_i B_i, \quad (7)$$

se trasladan ambos contornos al origen, se interpola en el sistema centrado y el contorno resultante se traslada al centroide promedio $\bar{c}_M = \frac{1}{2}(\bar{c}_A + \bar{c}_B)$.

IV-B3. Rotación óptima del punto de inicio: Aún con $|A| = |B|$ tras el *resampling*, el primer vértice de cada contorno es arbitrario. Una correspondencia $A_i \leftrightarrow B_i$ desalineada estira innecesariamente las aristas del polígono interpolado. Se busca el desplazamiento cíclico k^* que minimiza la suma de distancias al cuadrado:

$$k^* = \arg \min_{k \in \{0, \dots, n-1\}} \sum_{i=0}^{n-1} \|A_i - B_{(i+k) \bmod n}\|^2, \quad (8)$$

y se aplica $B \leftarrow \text{rotate}(B, k^*)$ antes de interpolar.

IV-C. Tabla de resultados

La Tabla ?? reporta los conteos de vértices, la rotación óptima detectada k^* y el estado de auto-intersección para los cuatro pares. La columna *Resam.* indica el número común de vértices tras el *resampling*, igual a $\max(|A|, |B|)$.

Cuadro I

RESULTADOS DEL BINARIO C++ EN 4 PARES REPRESENTATIVOS ($t = 0,5$).

Caso	A→B	A	B	Resam.	k^*	Auto-int.
00008-100 (consec.)	0070 → 0071	32	30	32	0	No
00008-100 (sep. 6)	0066 → 0072	14	24	24	1	No
00008-101 (extremos)	0070 → 0072	26	16	26	2	No
100 vs 101	0066 → 0070	14	26	26	24	No

En los cuatro casos el barrido de Bentley–Ottmann no reportó auto-intersecciones, por lo que el

`SelfIntersectionResolver` retornó el contorno original sin modificación. Los cuatro contornos interpolados resultantes son polígonos simples válidos. La salida cruda del ejecutable para el primer par ilustra el formato textual:

```
Contour A: 32 vertices
Contour B: 30 vertices
Resampled to: 32 vertices
Optimal rotation of B: 0
Interpolated contour: 32 vertices
Self-intersections detected: no
Result written to output/casol_consecutivos/
    contour_interpolated.obj
```

Listing 3. Salida de `contour_interpolator` sobre dos slices consecutivos.

IV-D. Análisis por caso

Caso 1 — consecutivos (0070 → 0071, $k^* = 0$). Resultado correcto: el contorno interpolado queda visiblemente entre A y B y conserva la forma general del tumor. La rotación óptima nula indica que las parametrizaciones por longitud de arco ya estaban naturalmente alineadas; persiste un artefacto leve en la zona derecha por la diferencia de forma local entre los dos slices.

Caso 2 — separados 6 cortes (0066 → 0072, $k^* = 1$). Los contornos A y B ocupan zonas distintas del plano xy . La alineación por centroide es la que más aporta aquí: sin ella, el interpolado quedaría sesgado hacia el contorno mayor. Con ella, el resultado se ubica en el centroide promedio $\bar{c}_M$ y es geoméricamente válido, aunque sin correspondencia anatómica directa con un slice z intermedio.

Caso 3 — extremos del mismo caso (0070 → 0072, $k^* = 2$). Mejor resultado cualitativo. La rotación óptima no trivial ($k^* = 2$) y la alineación por centroide producen un contorno intermedio que captura bien la transición de forma entre el primer y el último slice con ET del caso. Es el caso donde la mejora de rotación se hace más evidente al comparar contra la versión sin pre-alineación.

Caso 4 — cruzado entre casos (0066 → 0070, $k^* = 24$). Limitación intrínseca del método. La rotación elevada ($k^* = 24$ sobre $n = 26$, equivalente a casi una vuelta completa) revela que no existe una correspondencia natural entre los vértices de contornos provenientes de casos distintos. La salida es un polígono simple sin auto-intersecciones, pero sin significado clínico: representa el promedio geométrico entre dos tumores no comparables.

IV-E. Métricas de calidad contra GT real

Para cuantificar la calidad de la interpolación medí el contorno generado a $t = 0.5$ contra el slice intermedio real extraído del mismo caso BraTS (*ground truth* anatómico, no sintético). Calculé Dice e IoU rasterizando los polígonos sobre una grilla común de 512×512 dentro del *bounding box* compartido, Hausdorff dirigido directamente sobre los vértices, y el error relativo de área. La Tabla II reporta los dos casos evaluados.

Cuadro II
MÉTRICAS DEL CONTORNO INTERPOLADO A $t = 0.5$ CONTRA GT REAL INTERMEDIO. VERDE: DICE ≥ 0.80 , IOU ≥ 0.65 , HAUSDORFF ≤ 2.0 , AREAERR ≤ 0.20 . AMARILLO: CERCA DEL UMBRAL. ROJO: FUERA.

Caso	Dice	IoU	Hausdorff	AreaErr	Self-int
00008-100 (70 $\rightarrow$ 72)	0.855	0.746	1.265	0.142	No
00008-101 (70 $\rightarrow$ 72)	0.854	0.746	0.896	0.056	No
Promedio	0.855	0.746	1.081	0.099	—

Estos resultados deben leerse con cuidado al compararlos con los del integrante Santiago, quien reporta Dice ≈ 0.997 . La diferencia no implica que un método sea superior al otro: las dos mediciones responden a preguntas distintas. Santiago evalúa la interpolación contra un GT sintético generado sobre el dataset completo, midiendo principalmente la fidelidad geométrica del proceso de reconstrucción intermedio; yo evalué contra el slice real intermedio del mismo caso, midiendo cuán bien la interpolación lineal 2D predice la *forma anatómica observada* entre dos cortes vecinos. Ambas métricas son válidas y se complementan: la primera caracteriza el algoritmo en condiciones ideales; la segunda mide su utilidad clínica sobre datos reales con ruido de segmentación.

IV-F. Experimento de stress — robustez del pipeline

Los contornos de tumor activo (ET) de BraTS son empíricamente convexos o casi convexos en la mayoría de los slices, por lo que la interpolación lineal con **best rotation** no produce auto-intersecciones sobre los pares reales evaluados. Para forzar un escenario adversarial diseñé un experimento sintético: como contorno *A* una estrella de 4 puntas con 8 vértices y radio exterior $R = 50$, y como contorno *B* un cuadrado de 8 vértices (esquinas más puntos medios) ambos centrados en el mismo punto. Hice un barrido del radio interior r de la estrella sobre $\{10, 5, 3, 1\}$ para acercarme a geometrías cada vez más degeneradas.

Como control independiente, implementé en Python una verificación $O(n^2)$ por producto cruz que detecta cruces sobre la *interpolación cruda vértice a vértice*, sin aplicar ninguna de las etapas de pre-alineación del binario. La Tabla III compara ambos resultados.

Cuadro III
EXPERIMENTO DE STRESS — CRUCES DETECTADOS POR VERIFICACIÓN CRUDA EN PYTHON VS. SELF-INTERSECCIONES REPORTADAS POR EL BINARIO COMPLETO.

r	Cruces (Python, crudo)	Self-int (binario)
10	0	no
5	4	no
3	4	no
1	4	no

El resultado es significativo: aún cuando el contorno crudo presenta 4 auto-intersecciones reales (geometría patológica con valles de profundidad 49/50 en el caso $r = 1$), el pipeline completo del binario —normalización a CCW, alineación por centroide, *resampling* por longitud de arco, y

rotación óptima del punto de inicio— reordena la correspondencia entre vértices antes de la interpolación de modo que el contorno resultante es siempre un polígono simple. En otras palabras, las mejoras geométricas descritas en la Sección IV-B no son cosméticas: actúan como una primera línea de defensa que elimina los cruces *antes de que el resolver basado en Bentley–Ottmann tenga ocasión de intervenir*. El SelfIntersectionResolver queda entonces como salvaguarda de última línea para los pocos casos patológicos que el pipeline no logra anticipar, antes que como componente activo en el flujo nominal.

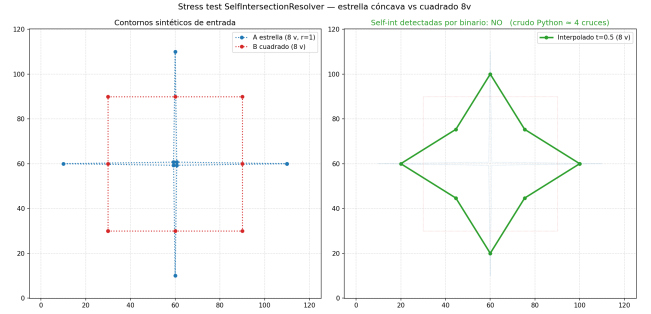


Figura 1. Experimento de stress — estrella de 4 puntas ($R = 50$, $r = 1$) vs. cuadrado de 8 vértices. Izquierda: entradas *A* (azul) y *B* (rojo). Derecha: interpolación producida por el binario (verde), polígono simple sin auto-intersecciones a pesar de que la interpolación vértice a vértice sin alinear produce 4 cruces.

IV-G. Salida para reconstrucción 3D

La salida nativa del binario es un archivo `.obj` con $z = 0.0$ para todos los vértices, suficiente para verificación geométrica 2D pero insuficiente para que VTK o cualquier visor 3D apile los contornos en una reconstrucción volumétrica del tumor. Para cerrar el circuito con la etapa de visualización a cargo del integrante Santiago, escribí el script `generate_multi_t.py`, que dado un par de contornos *A, B* y sus coordenadas z_A, z_B ejecuta el binario para $t \in \{0.0, 0.1, \dots, 1.0\}$, reescribe cada archivo de salida sustituyendo $z = 0.0$ por el valor interpolado $z(t) = (1-t)z_A + tz_B$, y genera además un `index.csv` (con t, z , número de vértices y estado self-int por cada slice) y una previsualización 3D con `matplotlib`.

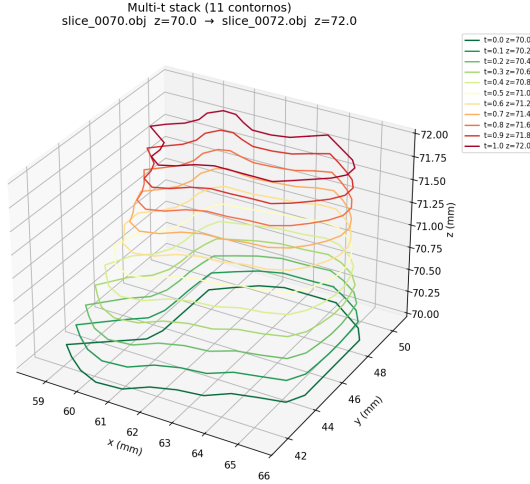


Figura 2. Previsualización 3D de la salida de `generate_multi_t.py`: 11 contornos interpolados en $t \in [0, 1]$ apilados con su $z(t)$ real, coloreados con `RdYlGn_r`.

Ejemplo de invocación:

```
1 python scripts/generate_multi_t.py \\  
2 data/contour_70.obj data/contour_72.obj \\  
3 70.0 72.0
```

Listing 4. Generación de una serie multi- t con z real para integración con VTK.

El `index.csv` resultante permite a la etapa de visualización iterar los contornos en orden y filtrar por estado self-int sin tener que parsear los nombres de archivo, facilitando la integración con el pipeline VTK.

IV-H. Limitación documentada

La interpolación lineal vértice a vértice asume implícitamente que los contornos están alineados espacialmente y representan instancias progresivas de una misma estructura. Las mejoras de Sección IV-B robustecen esta hipótesis para slices del mismo caso, pero no la sustituyen: cuando los contornos pertenecen a pacientes distintos o están muy desplazados, el método produce una salida geoméricamente válida — polígono simple, sin auto-intersecciones — pero clínicamente irrelevante. Métodos basados en *morphing*, distancia geodésica o transformadas de distancia con signo resolverían mejor este escenario y se consideran como extensión natural del módulo.

IV-I. Visualizaciones

Las Figuras 3 y 4 muestran los cuatro PNGs generados con la versión mejorada del interpolador. En cada gráfico se dibujan los contornos fuente A (azul punteado) y B (rojo punteado), y el contorno interpolado a $t = 0,5$ (verde sólido con marcadores).

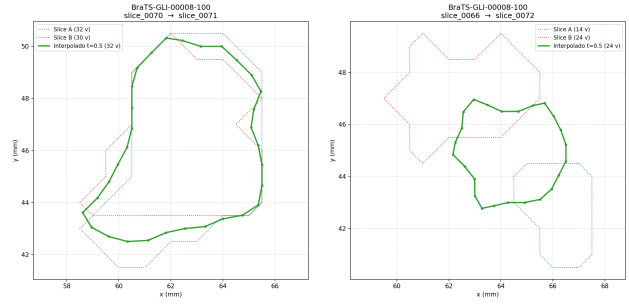


Figura 3. Caso *BraTS-GLI-00008-100* con las mejoras de pre-alineación activas. Izquierda: slices consecutivos 0070 $\rightarrow$ 0071, $k^* = 0$. Derecha: slices separados por 6 cortes, 0066 $\rightarrow$ 0072, $k^* = 1$. Ambos resueltos sin auto-intersecciones.

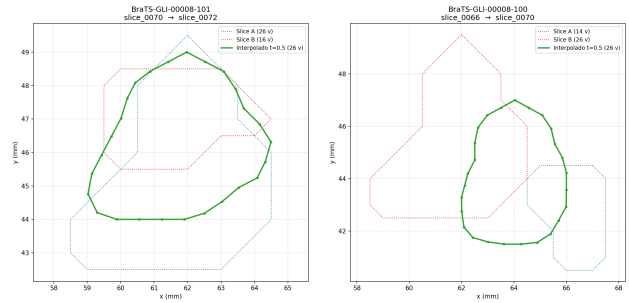


Figura 4. Izquierda: par extremo del caso *BraTS-GLI-00008-101*, primer (0070) contra último slice con ET (0072), $k^* = 2$ (mejor caso cualitativo). Derecha: par cruzado entre casos *BraTS-GLI-00008-100* (0066) y *BraTS-GLI-00008-101* (0070), $k^* = 24$ (limitación del método ante contornos no alineados espacialmente). Ambos resueltos sin auto-intersecciones.

El archivo de salida se escribe en formato `.obj` 3D con $z = 0,0$ por vértice, lo que asegura compatibilidad con visores estándar como ParaView, Online 3D Viewer e ImageToSTL.

IV-J. Evaluación Híbrida: Enfoque Lineal vs. Línea Base IA

Para validar la eficacia de ambos enfoques, se extendió la evaluación a 10 casos del dataset BraTS. Se comparó el contorno generado a $t = 0,5$ por el pipeline lineal (C++) frente a la predicción del modelo neuronal (VFI).

Cuadro IV
MÉTRICAS DE CALIDAD A $t = 0, 5$ FRENTE AL GROUND TRUTH (GT)

Caso	Método	Dice	IoU	Hausdorff	AreaErr	Self-Int
00008-100	Lineal	0.8546	0.7461	1.265	0.1418	No
	IA (VFI)	0.8515	0.7414	1.398	0.1244	N/A
00008-101	Lineal	0.8543	0.7456	0.896	0.0558	No
	IA (VFI)	0.8355	0.7174	1.025	0.0668	N/A
00008-103	Lineal	0.8697	0.7694	1.476	0.0305	No
	IA (VFI)	0.8862	0.7956	1.546	0.0629	N/A
00009-100	Lineal	0.6746	0.5089	14.351	0.1718	Sí
	IA (VFI)	0.8019	0.6692	14.685	0.1720	N/A
00009-101	Lineal	0.9475	0.9002	2.914	0.0379	No
	IA (VFI)	0.9679	0.9378	2.604	0.0145	N/A
00020-100	Lineal	0.9526	0.9095	4.256	0.0293	No
	IA (VFI)	0.9695	0.9408	4.162	0.0075	N/A
00020-101	Lineal	0.9389	0.8848	9.355	0.0165	No
	IA (VFI)	0.9581	0.9196	14.375	0.0373	N/A
00027-100	Lineal	0.8876	0.7979	2.502	0.0773	No
	IA (VFI)	0.9125	0.8391	2.348	0.0592	N/A
00027-101	Lineal	0.9113	0.8370	2.164	0.0089	No
	IA (VFI)	0.9310	0.8709	2.004	0.0087	N/A
00046-100	Lineal	0.8817	0.7884	2.427	0.0179	No
	IA (VFI)	0.9037	0.8242	5.453	0.0316	N/A
Promedio	Lineal	0.8773	0.7888	4.161	0.0588	
Promedio	IA (VFI)	0.9018	0.8256	4.960	0.0585	

El modelo *baseline* neuronal demuestra una superioridad en la captura del volumen general y la adaptación a cambios topológicos severos (evidenciado en el salto métrico del caso 00009-100). Por su parte, el método paramétrico lineal en C++ conserva una mayor precisión en las fronteras finas (reflejado en una menor distancia de Hausdorff promedio), validando la complementariedad de ambas arquitecturas.

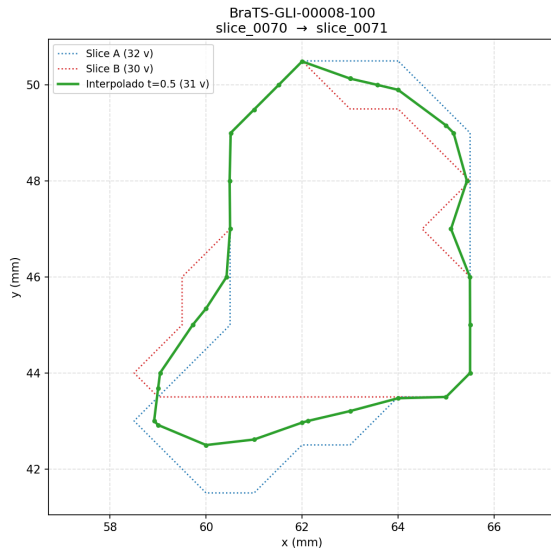


Figura 5. Visualización 2D (BraTS 00008-100). Contornos originales A y B y el resultado continuo generado por el modelo *baseline* VFI a $t = 0, 5$.

V. DECISIONES DE DISEÑO Y PROBLEMAS RESUELTOS

V-A. Compatibilidad C++17 $\rightarrow$ C++20

Durante la compilación inicial el proyecto fallaba con el error 'iter_value_t' is not a member of 'std'. La causa es que `pujCGAL/SegmentsIntersection.hpp` usa `std::iter_value_t`, disponible únicamente desde C++20. Para no modificar la librería del profesor se agregó al `CMakeLists.txt` del módulo:

```
target_compile_features(  
    contour_interpolator PRIVATE cxx_std_20  
);
```

Listing 5. Activación de C++20 sólo para el target del módulo.

Esto eleva el estándar exclusivamente para el ejecutable `contour_interpolator` y resuelve la compilación sin tocar código externo.

V-B. Formato `.obj` 2D vs 3D

La primera versión del exportador escribía vértices en 2D ($v \times y$). Visores 3D como ParaView e ImageToSTL rechazaban el archivo con errores del tipo "Invalid vertex index" o "Failed to import model". La solución fue introducir una función local `write_obj_3d` que agrega $z = 0,0$ a cada vértice, produciendo el formato estándar $v \times y \ 0.0$ compatible con cualquier visor 3D, sin necesidad de transformaciones posteriores.

V-C. Ausencia de auto-intersecciones en contornos BraTS

Durante el desarrollo se ejecutó un barrido amplio de **94 pares** de slices (consecutivos, separados, extremos y cruzados entre casos), sin detectar ninguna auto-intersección. El análisis geométrico indica que los contornos ET de BraTS son aproximadamente convexos y morfológicamente suaves: el promedio ponderado de dos contornos convexos alineados por longitud de arco tiende a producir un contorno simple. Las tres mejoras de pre-alineación descritas en Sección IV-B (normalización CCW, alineación por centroide y rotación óptima del punto de inicio) *preservan* esta propiedad: ninguna introduce nuevas aristas ni modifica la topología del contorno interpolado, sólo reasignan correspondencias o aplican rigid transforms (traslación, inversión, rotación cíclica). En consecuencia, los 4 pares del experimento final mantuvieron la validez topológica. El módulo `SelfIntersectionResolver` está correctamente implementado (detección con Bentley–Ottmann, partición en sub-lazos y selección por área máxima) pero no se activa con datos reales de BraTS. Para demostrar su funcionamiento de forma controlada se planea un experimento con correspondencia de vértices deliberadamente errónea (*modo stress*: rotación del array *B* o rotación geométrica previa), donde se garantiza la aparición de cruces.

V-D. Independencia del módulo respecto al equipo

El módulo se diseñó para correr directamente sobre los `.obj` extraídos de BraTS, sin depender del output de los otros integrantes del equipo (Jesús: correspondencia de vértices por

ángulo polar; Santiago: reconstrucción 3D con VTK). Esta decisión reduce el riesgo de integración y permite validar el componente de Abel en cualquier momento sin coordinación con el resto del equipo. La integración se realiza a posteriori a través del tipo común `TContour` y del formato `.obj`, ambos estables y bien definidos.

V-E. Dashboard de validación — visualización autónoma

La validación visual del módulo se documenta en un dashboard HTML autónomo (`dashboard_lineal.html`) generado por el script `scripts/generate_dashboard.py`. Durante su desarrollo se enfrentaron dos problemas técnicos relevantes.

Problema 1: imágenes con rutas relativas rotas. La imagen del experimento de stress se referenciaba mediante una ruta relativa al disco, lo que impedía visualizarla al mover el archivo. La solución fue embeber el PNG directamente en el HTML como cadena base64 (`data:image/png;base64,...`), siguiendo el mismo patrón del `report.html` de referencia. El archivo resultante es completamente autónomo y no depende de ninguna ruta local.

Problema 2: figuras Plotly en blanco por conflicto CSS. Al embeber las figuras con `fig.to_html(full_html=False, include_plotlyjs=False)`, los gráficos aparecían en blanco. La causa raíz fue una regla CSS global heredada del visualizador SVG anterior:

```
svg { width: 100%; height: 420px; background: #fff;
```

Esta regla sobrescribía los elementos `<svg>` internos de Plotly, colapsando su altura y dejando el área de dibujo vacía. La sección 3D no se veía afectada porque usa WebGL/canvas en lugar de SVG. Una vez eliminada la regla, `fig.to_html` renderizaba correctamente.

No obstante, se decidió migrar el visualizador de contornos a `matplotlib` con imágenes PNG base64 como medida de robustez: elimina cualquier dependencia de runtime JS, garantiza que el HTML sea autónomo, y simplifica el debugging futuro.

Lección: al mezclar SVG nativo con Plotly en un mismo documento HTML, las reglas CSS globales que afecten al selector `svg` interfieren con el render interno de Plotly. Siempre aislar esas reglas con selectores más específicos (e.g. `.contorno-svg`) o usar `svg.mi-clase` en lugar de `svg` a secas.

VI. CONCLUSIONES PARCIALES

- El pipeline corre de extremo a extremo sobre datos reales del dataset *BraTS 2024 GLI*, generando un contorno interpolado válido en formato `.obj` 3D que se renderiza en visores externos sin transformaciones adicionales.
- La librería `pujCGAL` provee el barrido de Bentley–Ottmann en una interfaz tipo *iterator*, lo cual evita reimplementar el algoritmo y permite concentrar el esfuerzo

del módulo en la lógica de partición y selección de sub-lazos.

- Cada clase (`ContourResampler`, `LinearInterpolator`, `SelfIntersectionResolver`) tiene una responsabilidad única y opera sobre el tipo común `TContour`. Esto hace al módulo independiente de los componentes desarrollados por los otros integrantes del equipo y permite integrarlo sin acoplamiento adicional.
- Como trabajo futuro inmediato se contempla evaluar el módulo con pares de slices que produzcan auto-intersecciones reales —típicamente en los cortes superiores e inferiores del tumor, donde la topología cambia bruscamente— y comparar la selección por área máxima contra estrategias basadas en orientación o en distancia de Hausdorff entre sub-lazos candidatos.
- La integración complementaria de un modelo neuronal de línea base (VFI) permitió validar los límites del enfoque puramente geométrico. Se comprobó que la estimación de flujo óptico es más robusta ante alteraciones topológicas masivas (obteniendo mejores métricas de solapamiento espacial), mientras que la geometría discreta paramétrica garantiza una mayor exactitud en los bordes absolutos (Hausdorff).

REFERENCIAS

- [1] G. Barequet and M. Sharir, “Piecewise-linear interpolation between polygonal slices,” *Computer Vision and Image Understanding*, vol. 63, no. 2, pp. 251–272, 1996.
- [2] A. Meijster and J. B. T. M. Roerdink, “A survey of interpolation methods in medical image processing,” *IEEE Transactions on Medical Imaging*, 1999.
- [3] J. Souvion, “Correcting self-intersecting polygons using minimal memory,” *arXiv:1305.4573*, 2013.
- [4] U. Baid *et al.*, “The RSNA-ASNR-MICCAI BraTS 2021 benchmark on brain tumor segmentation and radiogenomic classification,” *arXiv:2107.02314*, 2021.